

# Code Similarity Platform

## Issue Resolution Procedures

**Version:** 1.0

**Prepared by:** Group 16 — Muhammad Nabeel Waheed, Thomas Neal,  
Ghassan Balouze, Kev Akpinar, Abdel Zahran, Ty Mabee

**Kool Group Name:** Last Minute Warriors

**Course:** COSC 4P02 — Course Project

2026-04-24

# Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Incident Response Overview</b>                  | <b>3</b>  |
| 1.1      | Severity Classification . . . . .                  | 3         |
| 1.2      | General First-Response Checklist . . . . .         | 3         |
| 1.3      | Escalation Matrix Template . . . . .               | 4         |
| <b>2</b> | <b>Diagnostic Toolkit</b>                          | <b>5</b>  |
| 2.1      | Standard Diagnostic Commands . . . . .             | 5         |
| 2.1.1    | Docker and Container State . . . . .               | 5         |
| 2.1.2    | PostgreSQL Diagnostics . . . . .                   | 5         |
| 2.1.3    | Redis Diagnostics . . . . .                        | 6         |
| 2.1.4    | Object Storage Diagnostics . . . . .               | 7         |
| 2.1.5    | Engine Diagnostics . . . . .                       | 7         |
| 2.2      | Checking Connectivity Between Components . . . . . | 8         |
| 2.3      | Assessing Resource Usage . . . . .                 | 8         |
| <b>3</b> | <b>Component-by-Component Runbooks</b>             | <b>10</b> |
| 3.1      | API Server Runbook . . . . .                       | 10        |
| 3.1.1    | Common Failure Symptoms . . . . .                  | 10        |
| 3.1.2    | Diagnostic Procedure . . . . .                     | 10        |
| 3.1.3    | Resolution Steps . . . . .                         | 11        |
| 3.1.4    | Verification Steps . . . . .                       | 11        |
| 3.1.5    | Rollback Procedure . . . . .                       | 11        |
| 3.2      | Background Worker Runbook . . . . .                | 12        |
| 3.2.1    | Common Failure Symptoms . . . . .                  | 12        |
| 3.2.2    | Diagnostic Procedure . . . . .                     | 12        |
| 3.2.3    | Resolution Steps . . . . .                         | 13        |
| 3.2.4    | Verification Steps . . . . .                       | 13        |
| 3.2.5    | Rollback Procedure . . . . .                       | 14        |
| 3.3      | Analysis Engine Runbook . . . . .                  | 14        |
| 3.3.1    | Common Failure Symptoms . . . . .                  | 14        |
| 3.3.2    | Diagnostic Procedure . . . . .                     | 14        |
| 3.3.3    | Resolution Steps . . . . .                         | 15        |
| 3.3.4    | Verification . . . . .                             | 15        |
| 3.4      | PostgreSQL Runbook . . . . .                       | 15        |
| 3.4.1    | Common Failure Symptoms . . . . .                  | 15        |
| 3.4.2    | Diagnostic Procedure . . . . .                     | 16        |
| 3.4.3    | Resolution Steps . . . . .                         | 16        |
| 3.4.4    | Verification . . . . .                             | 17        |
| 3.5      | Redis Runbook . . . . .                            | 18        |
| 3.5.1    | Common Failure Symptoms . . . . .                  | 18        |
| 3.5.2    | Diagnostic Procedure . . . . .                     | 18        |
| 3.5.3    | Resolution Steps . . . . .                         | 18        |
| 3.5.4    | Verification . . . . .                             | 19        |

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| 3.6      | Frontend (Next.js) Runbook . . . . .                                    | 19        |
| 3.6.1    | Common Failure Symptoms . . . . .                                       | 19        |
| 3.6.2    | Diagnostic Procedure . . . . .                                          | 19        |
| 3.6.3    | Resolution Steps . . . . .                                              | 19        |
| <b>4</b> | <b>Common Cross-Cutting Issues</b>                                      | <b>20</b> |
| 4.1      | Service Startup Failures . . . . .                                      | 20        |
| 4.2      | Database Connection Failures or Pool Exhaustion . . . . .               | 20        |
| 4.3      | Authentication / Authorization Failures . . . . .                       | 21        |
| 4.4      | Performance Degradation or Timeouts . . . . .                           | 21        |
| 4.5      | Memory Leaks or Resource Exhaustion . . . . .                           | 22        |
| 4.6      | Failed Background Jobs or Scheduled Tasks . . . . .                     | 22        |
| 4.7      | Data Corruption or Inconsistency Scenarios . . . . .                    | 23        |
| 4.8      | Configuration Errors . . . . .                                          | 24        |
| 4.9      | Dependency / Integration Failures (Third-Party Services Down) . . . . . | 25        |
| 4.10     | Deployment Failures and Rollback . . . . .                              | 25        |
| <b>5</b> | <b>Data Recovery Procedures</b>                                         | <b>26</b> |
| 5.1      | Full Restore from Backup . . . . .                                      | 26        |
| 5.2      | Point-in-Time Recovery . . . . .                                        | 26        |
| 5.3      | Partial Data Recovery . . . . .                                         | 27        |
| <b>6</b> | <b>Post-Incident Procedures</b>                                         | <b>28</b> |
| 6.1      | Root Cause Analysis Template . . . . .                                  | 28        |
| 6.2      | Incident Report . . . . .                                               | 29        |
| 6.3      | Implementing Preventive Fixes . . . . .                                 | 29        |

# Chapter 1. Incident Response Overview

## 1.1 Severity Classification

| Severity        | Definition                                                                                           | Examples                                                                                                           |
|-----------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Critical</b> | Complete service unavailability or data loss risk. Requires immediate response regardless of time.   | All users unable to log in; database unreachable; object storage data loss; security breach.                       |
| <b>High</b>     | Core functionality broken for a significant subset of users or a time-sensitive workflow is blocked. | Comparison runs failing for all assignments; upload processing queue stuck; API returns 500 on most requests.      |
| <b>Medium</b>   | Degraded functionality; workaround exists or only some users/workflows affected.                     | Specific upload batch stuck in PROCESSING; single comparison run failed; identity reveal fails for one submission. |
| <b>Low</b>      | Cosmetic issue, minor feature misbehaviour, no data loss.                                            | UI display glitch; slow response times within acceptable SLAs.                                                     |

## 1.2 General First-Response Checklist

For any incident, before deeper investigation:

1. Confirm the incident is real: reproduce or verify with a second data source.
2. Classify severity (section above).
3. Check service health endpoints:

```
1 curl -s https://similarityapp.ca/api/health/live
2 curl -s https://similarityapp.ca/api/health/ready
```

4. Check all Docker services are running:

```
1 docker compose ps
```

5. Check recent logs for each service:

```
1 docker compose logs --tail=100 api
2 docker compose logs --tail=100 worker
```

```
3 docker compose logs --tail=100 postgres
4 docker compose logs --tail=100 redis
```

6. Check disk space:

```
1 df -h
2 docker system df
```

7. Record timeline, symptoms, and initial findings.

## 1.3 Escalation Matrix Template

| Severity | Response time                   | Escalation path                                                   |
|----------|---------------------------------|-------------------------------------------------------------------|
| Critical | Immediate (24/7)                | On-call engineer →<br>System administrator →<br>Course instructor |
| High     | Within 2 hours (business hours) | On-call engineer → Sys-<br>tem administrator                      |
| Medium   | Within 24 hours                 | On-call engineer                                                  |
| Low      | Next business day               | Assigned engineer                                                 |

**NOTE:** No formal on-call rotation is defined in the repository. This template should be populated with actual contact information before production use.

## Chapter 2. Diagnostic Toolkit

### 2.1 Standard Diagnostic Commands

#### 2.1.1 Docker and Container State

Listing 2.1: Container diagnostics

```
1 # All service status  
2 docker compose ps  
3  
4 # Resource usage (CPU, memory) per container  
5 docker stats --no-stream  
6  
7 # Inspect a specific container's configuration  
8 docker compose inspect api  
9  
10 # Check if a container has exited and why  
11 docker compose logs api | tail -50  
12 docker inspect $(docker compose ps -q api) | jq '.[0].State'  
13  
14 # Restart a specific service  
15 docker compose restart api  
16  
17 # Rebuild and restart one service  
18 docker compose up -d --no-deps --build api
```

#### 2.1.2 PostgreSQL Diagnostics

Listing 2.2: PostgreSQL diagnostics

```
1 # Check connectivity  
2 docker compose exec postgres pg_isready -U postgres  
3  
4 # Open psql shell  
5 docker compose exec postgres psql -U postgres -d similarity_app  
6  
7 # Active connections  
8 psql $DATABASE_URL -c \  
9     "SELECT pid, state, query_start, LEFT(query, 80) AS query  
10     FROM pg_stat_activity  
11     WHERE datname = 'similarity_app'  
12     ORDER BY query_start DESC;"  
13  
14 # Long-running queries (> 30 seconds)  
15 psql $DATABASE_URL -c \  
16     "SELECT pid, now() - query_start AS duration, state, LEFT(query  
    , 120) AS query
```

```

17     FROM pg_stat_activity
18     WHERE state != 'idle'
19           AND now() - query_start > interval '30 seconds'
20     ORDER BY duration DESC;"
21
22 # Table sizes
23 psql $DATABASE_URL -c \
24     "SELECT schemaname, tablename,
25           pg_size_pretty(pg_total_relation_size(quote_ident(tablename)
26           )) AS total_size
27     FROM pg_tables
28     WHERE schemaname = 'public'
29     ORDER BY pg_total_relation_size(quote_ident(tablename)) DESC;"
30
31 # Row counts per table
32 psql $DATABASE_URL -c \
33     "SELECT relname, n_live_tup FROM pg_stat_user_tables ORDER BY
34     n_live_tup DESC;"
35
36 # Check for locks
37 psql $DATABASE_URL -c \
38     "SELECT pid, relation::regclass, mode, granted FROM pg_locks
39     WHERE NOT granted;"

```

### 2.1.3 Redis Diagnostics

Listing 2.3: Redis diagnostics

```

1 # Ping
2 docker compose exec redis redis-cli PING
3
4 # Memory usage
5 docker compose exec redis redis-cli INFO memory
6
7 # BullMQ queue stats
8 # Queue names: upload-preparation, comparison-run
9 for queue in upload-preparation comparison-run; do
10     echo "=== $queue ==="
11     docker compose exec redis redis-cli LLEN bull:$queue:wait
12     docker compose exec redis redis-cli LLEN bull:$queue:active
13     docker compose exec redis redis-cli LLEN bull:$queue:failed
14     docker compose exec redis redis-cli LLEN bull:$queue:completed
15 done
16
17 # List failed job IDs
18 docker compose exec redis redis-cli LRange bull:comparison-run:
19     failed 0 9
20
21 # Get a specific failed job's data
22 docker compose exec redis redis-cli HGET bull:comparison-run:<
23     jobId> data

```

```

22
23 # Total key count
24 docker compose exec redis redis-cli DBSIZE

```

## 2.1.4 Object Storage Diagnostics

Listing 2.4: Object storage diagnostics

```

1 # Check disk usage
2 docker run --rm \
3   -v similarity-app_object-storage:/storage:ro \
4   busybox du -sh /storage
5
6 # List top-level directories
7 docker run --rm \
8   -v similarity-app_object-storage:/storage:ro \
9   busybox ls -la /storage
10
11 # Count files
12 docker run --rm \
13   -v similarity-app_object-storage:/storage:ro \
14   busybox find /storage -type f | wc -l
15
16 # Find orphaned staging files (files in _staging/ older than 1
17   hour)
18 docker run --rm \
19   -v similarity-app_object-storage:/storage:ro \
20   busybox find /storage/_staging -type f -mmin +60

```

## 2.1.5 Engine Diagnostics

Listing 2.5: Engine binary diagnostics

```

1 # Verify engine binary exists and is executable
2 docker compose exec worker ls -la /app/bin/engine-cli
3
4 # Quick smoke-test: pipe minimal valid request
5 docker compose exec worker sh -c 'echo '''{
6   "schemaVersion": "1.0",
7   "engineVersion": "test",
8   "language": "java",
9   "submissions": [
10     {"submissionId": "a", "submissionKind": "current",
11      "source": "class A {}\n", "sourceMap": []},
12     {"submissionId": "b", "submissionKind": "current",
13      "source": "class B {}\n", "sourceMap": []}
14   ],
15   "pairs": [{"pairId": "a_b", "leftSubmissionId": "a", "
16     rightSubmissionId": "b"}],
17   "params": {"gstMinMatchLength": 8, "minimumCommentLength": 12}
18 }''' | /app/bin/engine-cli'

```



```
18 # Should return valid JSON response with similarityScore: 0.0
19
20 # Check engine exit code
21 echo $? # Should be 0
```

## 2.2 Checking Connectivity Between Components

Listing 2.6: Inter-component connectivity checks

```
1 # From API container: can reach PostgreSQL?
2 docker compose exec api sh -c \
3   'nc -zv postgres 5432 && echo OK || echo FAILED'
4
5 # From API container: can reach Redis?
6 docker compose exec api sh -c \
7   'nc -zv redis 6379 && echo OK || echo FAILED'
8
9 # From worker container: can reach PostgreSQL?
10 docker compose exec worker sh -c \
11   'nc -zv postgres 5432 && echo OK || echo FAILED'
12
13 # From worker container: can reach Redis?
14 docker compose exec worker sh -c \
15   'nc -zv redis 6379 && echo OK || echo FAILED'
16
17 # From Caddy: can reach API?
18 docker compose exec caddy sh -c \
19   'nc -zv api 3001 && echo OK || echo FAILED'
20
21 # From Caddy: can reach web?
22 docker compose exec caddy sh -c \
23   'nc -zv web 3000 && echo OK || echo FAILED'
```

## 2.3 Assessing Resource Usage

Listing 2.7: Resource usage assessment

```
1 # Real-time container resource usage
2 docker stats
3
4 # Host system resource usage
5 top -b -n 1 | head -20
6
7 # Disk usage breakdown
8 df -h
9 du -sh /var/lib/docker/volumes/
10
11 # PostgreSQL memory
12 docker compose exec postgres psql -U postgres -c \
```

```
13 "SELECT pg_size_pretty(pg_database_size('similarity_app')) AS  
    db_size;"  
14  
15 # Redis memory  
16 docker compose exec redis redis-cli INFO memory | grep -E '  
    used_memory_human|maxmemory_human'
```

## Chapter 3. Component-by-Component Runbooks

### 3.1 API Server Runbook

#### 3.1.1 Common Failure Symptoms

| Symptom                              | Probable cause                                              |
|--------------------------------------|-------------------------------------------------------------|
| GET /health/live returns non-200     | API process crashed or not started                          |
| GET /health/ready returns 503        | Database or Redis unreachable                               |
| All requests return 401 Unauthorized | Session cookie domain/secure mismatch; all sessions expired |
| All requests return 500              | Uncaught exception; database connection failure             |
| File uploads fail with 413           | File exceeds <code>UPLOAD_MAX_FILE_SIZE_MB</code>           |
| CORS errors in browser               | <code>CORS_ALLOWED_ORIGINS</code> not set correctly         |
| API starts but immediately exits     | Missing required environment variable                       |

#### 3.1.2 Diagnostic Procedure

1. Check if the API container is running:

```
1 docker compose ps api
```

2. Check API logs for startup errors:

```
1 docker compose logs --tail=200 api
```

Look for: `PrismaClientInitializationError`, `is required`, `ECONNREFUSED`, `Error`.

3. Test health endpoints:

```
1 curl -v http://localhost:3001/health/live
2 curl -v http://localhost:3001/health/ready
```

4. Verify database connectivity from API container (Section 2.1).
5. Verify Redis connectivity from API container (Section 2.1).
6. Check environment variables are set:

```
1 docker compose exec api env | grep -E 'DATABASE_URL|REDIS_URL|
  NODE_ENV|API_PORT'
```

### 3.1.3 Resolution Steps

API process not running:

```
1 docker compose up -d api
2 docker compose logs -f api # Watch for successful startup
```

Missing environment variable:

```
1 # Identify the missing variable from logs, e.g.:
2 # Error: DATABASE_URL is required
3 # Fix: edit .env and restart
4 nano .env
5 docker compose up -d api
```

Database connection refused:

```
1 # Check postgres container is running
2 docker compose ps postgres
3 docker compose up -d postgres
4 # Wait for postgres to be healthy, then restart API
5 sleep 15 && docker compose restart api
```

CORS errors in browser:

```
1 # Verify the CORS_ALLOWED_ORIGINS matches the exact origin
2 # (including protocol and port) of the web frontend
3 grep CORS_ALLOWED_ORIGINS .env
4 # Update if needed, then:
5 docker compose up -d api
```

### 3.1.4 Verification Steps

```
1 curl -s http://localhost:3001/health/live | jq .
2 # Expected: {"status":"ok","service":"api"}
3
4 curl -s http://localhost:3001/health/ready | jq .
5 # Expected: {"status":"ok","checks":{"database":"ok","redis":"ok"}}
```

### 3.1.5 Rollback Procedure

```
1 # Roll back to previous Docker image (if built with tags)
2 docker compose stop api
3 docker tag similarity-app_api:previous similarity-app_api:current
4 docker compose up -d api
5
6 # OR: check out previous Git tag and rebuild
7 git checkout <previous-tag>
8 docker compose up --build -d api
```

## 3.2 Background Worker Runbook

### 3.2.1 Common Failure Symptoms

| Symptom                                                           | Probable cause                                                  |
|-------------------------------------------------------------------|-----------------------------------------------------------------|
| Upload batches stuck in PROCESSING                                | Worker crashed; BullMQ job stuck active                         |
| Upload batches stay in RECEIVED                                   | Worker not running; Redis not reachable                         |
| Comparison runs stuck in QUEUED                                   | Worker not running; comparison queue not consumed               |
| Comparison runs stuck in RUNNING                                  | Worker crashed during run; engine binary issue                  |
| Worker starts then immediately exits                              | Missing <code>ENGINE_BINARY_PATH</code> ; binary not executable |
| <code>ARCHIVE_MAX_EXPANDED_BYTES</code> exceeded in error message | Uploaded archive is too large; increase limit or split archive  |

### 3.2.2 Diagnostic Procedure

1. Check worker container state:

```
1 docker compose ps worker
2 docker compose logs --tail=200 worker
```

2. Check Redis queue depths:

```
1 for q in upload-preparation comparison-run; do
2   echo "=== $q ==="
3   docker compose exec redis redis-cli LLEN bull:$q:wait
4   docker compose exec redis redis-cli LLEN bull:$q:active
5   docker compose exec redis redis-cli LLEN bull:$q:failed
6 done
```

3. Verify engine binary:

```
1 docker compose exec worker ls -la /app/bin/engine-cli
2 docker compose exec worker /app/bin/engine-cli 2>&1 | head -5
3 # Expect: JSON parse error or similar (binary is alive)
```

4. Check for stuck active jobs:

```
1 docker compose exec redis redis-cli LRange bull:comparison-run:
  active 0 9
```

5. Check upload batch statuses in database:

```
1 psql $DATABASE_URL -c \
2   "SELECT id, status, \"errorMessage\", \"createdAt\"
3   FROM \"UploadBatch\""
```

```

4 WHERE status IN ('RECEIVED', 'PROCESSING')
5 ORDER BY \"createdAt\" DESC LIMIT 10;"

```

### 3.2.3 Resolution Steps

Worker container crashed:

```

1 docker compose up -d worker
2 docker compose logs -f worker
3 # Healthy output: "BullMQ worker ready for queue: upload-
  preparation"

```

Stuck active job in Redis (worker crashed mid-job):

```

1 # Move stuck active jobs back to wait queue
2 # BullMQ will retry stalled jobs automatically when worker
  reconnects
3 # Check stalled check interval -- BullMQ default is 30 seconds
4
5 # Force-fail a stuck job (replace <jobId>):
6 docker compose exec redis redis-cli LREM bull:comparison-run:
  active 0 <jobId>
7 # Then mark the ComparisonRun as FAILED in the database:
8 psql $DATABASE_URL -c \
9   "UPDATE \"ComparisonRun\"
10    SET status = 'FAILED', \"errorMessage\" = 'Job forcibly
    recovered'
11    WHERE id = '<comparisonRunId>';"

```

Upload batch stuck in PROCESSING (worker crashed):

```

1 # Mark the batch as FAILED so professor is notified
2 psql $DATABASE_URL -c \
3   "UPDATE \"UploadBatch\"
4     SET status = 'FAILED',
5       \"errorMessage\" = 'Processing was interrupted. Please re-
      upload.'
6     WHERE status = 'PROCESSING';"
7
8 # Restart worker and advise professor to re-upload
9 docker compose restart worker

```

Engine binary missing or not executable:

```

1 # Rebuild worker image (includes engine build stage)
2 docker compose up --build -d worker

```

### 3.2.4 Verification Steps

```

1 # Confirm worker is consuming jobs
2 docker compose logs -f worker | grep -E "completed|failed|active"

```

```

3
4 # Check queue depths decrease over time
5 docker compose exec redis redis-cli LLEN bull:upload-preparation:
   wait
6 docker compose exec redis redis-cli LLEN bull:comparison-run:wait
7
8 # Verify a specific batch transitions to READY
9 psql $DATABASE_URL -c \
10     "SELECT id, status, \"updatedAt\" FROM \"UploadBatch\" ORDER BY
   \"updatedAt\" DESC LIMIT 5;"

```

### 3.2.5 Rollback Procedure

```

1 git checkout <previous-tag>
2 docker compose up --build -d worker

```

## 3.3 Analysis Engine Runbook

### 3.3.1 Common Failure Symptoms

| Symptom                                | Probable cause                                                      |
|----------------------------------------|---------------------------------------------------------------------|
| Comparison runs fail with engine error | Invalid input to engine; engine binary crash; parsing failure       |
| Engine returns empty pair results      | All submissions have zero source tokens (extraction failed earlier) |
| Very high engine run time              | Very large submissions; many pairs; low CPU                         |
| failed to parse Java/C/C++ source      | Source file is not valid syntax; tree-sitter parse error            |
| schema_version must be 1.0             | Worker and engine version mismatch                                  |

### 3.3.2 Diagnostic Procedure

1. Check the comparison run error message:

```

1 psql $DATABASE_URL -c \
2     "SELECT id, status, \"errorMessage\", \"engineVersion\", \"
   startedAt\", \"completedAt\"
3     FROM \"ComparisonRun\"
4     WHERE status = 'FAILED'
5     ORDER BY \"createdAt\" DESC LIMIT 5;"

```

2. Check worker logs for engine stderr output:

```

1 docker compose logs worker | grep -E "engine|Engine|error|
   stderr"

```

3. Run engine smoke-test (Section 2.4) to verify binary is functional.
4. Inspect submission source content for the failing assignment:

```

1 psql $DATABASE_URL -c \
2   "SELECT id, \"displayName\", kind, length(\"
   concatenatedSource\") AS src_len
3   FROM \"Submission\"
4   WHERE \"assignmentId\" = '<assignmentId>'
5   ORDER BY \"createdAt\" ASC;"

```

### 3.3.3 Resolution Steps

Engine parse failure (malformed source):

```

1 # Identify which submission has the problematic source
2 # (engine will report submissionId in error)
3 # Professor must re-upload corrected source files
4
5 # In the meantime, allow the comparison to run without that
   submission:
6 # Delete the problematic submission
7 curl -X DELETE \
8   -H "Cookie: similarity_session=<token>" \
9   https://similarityapp.ca/api/uploads/assignment/<id>/
   submissions/<submissionId>
10
11 # Then create a new comparison run

```

Schema version mismatch:

```

1 # Ensure worker and engine binary versions match
2 docker compose exec worker node -e \
3   "const c = require('./dist/config/runtime-config.js'); console.
   log(c.workerRuntimeConfig.engineVersion)"
4 docker compose exec worker /app/bin/engine-cli --version 2>&1 ||
   true
5 # Rebuild worker image to re-sync
6 docker compose up --build -d worker

```

### 3.3.4 Verification

After fixing: create a new comparison run for the same assignment and confirm it transitions to COMPLETED.

## 3.4 PostgreSQL Runbook

### 3.4.1 Common Failure Symptoms



| Symptom                                      | Probable cause                                            |
|----------------------------------------------|-----------------------------------------------------------|
| API/worker return 500 with PrismaClientError | Database connection refused or timed out                  |
| Readiness endpoint returns 503               | SELECT 1 fails                                            |
| Very slow queries                            | Missing index; table bloat; lock contention               |
| too many connections                         | Connection pool exhausted                                 |
| Database disk full                           | Excessive table growth; large blob data; WAL accumulation |

### 3.4.2 Diagnostic Procedure

Listing 3.1: PostgreSQL diagnosis

```

1  # Check if postgres container is running
2  docker compose ps postgres
3  docker compose logs postgres | tail -50
4
5  # Test connectivity
6  docker compose exec postgres pg_isready -U postgres
7
8  # Check connection count
9  psql $DATABASE_URL -c \
10     "SELECT count(*) AS active_connections FROM pg_stat_activity
11     WHERE datname = 'similarity_app';"
12
13 # Check for long-running or blocking queries
14 psql $DATABASE_URL -c \
15     "SELECT pid, now() - pg_stat_activity.query_start AS duration,
16     query, state, wait_event_type, wait_event
17     FROM pg_stat_activity
18     WHERE state != 'idle' AND datname = 'similarity_app'
19     ORDER BY duration DESC LIMIT 10;"
20
21 # Check disk usage
22 docker compose exec postgres df -h /var/lib/postgresql/data

```

### 3.4.3 Resolution Steps

PostgreSQL container not running:

```

1  docker compose up -d postgres
2  # Wait for it to become ready
3  until docker compose exec postgres pg_isready -U postgres; do
4      sleep 2; done
5  # Restart API and worker
6  docker compose restart api worker

```

Connection pool exhaustion:

```

1 # Increase max connections in postgres.conf
2 # (requires container restart)
3 docker compose exec postgres psql -U postgres \
4   -c "ALTER SYSTEM SET max_connections = 200;"
5 docker compose restart postgres
6
7 # OR: reduce Prisma connection pool size
8 # Add ?connection_limit=10 to DATABASE_URL in .env
9 # Example:
10 # DATABASE_URL=postgresql://...?schema=public&connection_limit=10
11 docker compose up -d api worker

```

### Blocking query / lock wait:

```

1 # Identify blocking PID
2 psql $DATABASE_URL -c \
3   "SELECT pid, query, state, wait_event_type FROM
4     pg_stat_activity
5     WHERE wait_event_type = 'Lock';"
6
7 # Terminate the blocking query (use with caution)
8 psql $DATABASE_URL -c "SELECT pg_terminate_backend(<pid>);"

```

### Disk space:

```

1 # Check what is consuming space
2 psql $DATABASE_URL -c \
3   "SELECT tablename,
4     pg_size_pretty(pg_total_relation_size(tablename::regclass))
5     AS size
6     FROM pg_tables WHERE schemaname = 'public'
7     ORDER BY pg_total_relation_size(tablename::regclass) DESC;"
8
9 # Run VACUUM to reclaim space
10 psql $DATABASE_URL -c "VACUUM ANALYZE;"
11
12 # If PairMatch is large: old comparison runs can be deleted
13 # (also deletes PairResult via cascade)
14 psql $DATABASE_URL -c \
15   "DELETE FROM \"ComparisonRun\"
16   WHERE \"completedAt\" < NOW() - INTERVAL '6 months'
17   AND status = 'COMPLETED';"

```

### 3.4.4 Verification

```

1 curl -s http://localhost:3001/health/ready | jq '.checks.database
2 # Expected: "ok"

```

## 3.5 Redis Runbook

### 3.5.1 Common Failure Symptoms

| Symptom                              | Probable cause                               |
|--------------------------------------|----------------------------------------------|
| API health check: redis "unexpected" | PING returned something other than PONG      |
| Upload/comparison jobs not queuing   | Redis connection refused from API            |
| Worker not processing jobs           | Redis connection refused from worker         |
| Redis OOM or eviction                | Memory limit reached; job payloads too large |

### 3.5.2 Diagnostic Procedure

```

1 docker compose ps redis
2 docker compose logs redis | tail -30
3 docker compose exec redis redis-cli PING
4 docker compose exec redis redis-cli INFO memory | grep -E '
  used_memory_human|maxmemory_human|mem_fragmentation_ratio'
```

### 3.5.3 Resolution Steps

Redis container not running:

```

1 docker compose up -d redis
2 until docker compose exec redis redis-cli PING | grep -q PONG; do
  sleep 2; done
3 docker compose restart api worker
```

Memory pressure / OOM:

```

1 # Check for old completed/failed jobs consuming memory
2 docker compose exec redis redis-cli LLEN bull:comparison-run:
  completed
3 docker compose exec redis redis-cli LLEN bull:upload-preparation:
  completed
4
5 # Clean up old completed jobs (BullMQ may be configured to keep N
  completed)
6 docker compose exec redis redis-cli DEL bull:comparison-run:
  completed
7 docker compose exec redis redis-cli DEL bull:upload-preparation:
  completed
8
9 # Set memory limit (add to docker-compose.yml redis service or
  redis.conf)
10 # command: redis-server --maxmemory 512mb --maxmemory-policy
  allkeys-lru
```

### 3.5.4 Verification

```
1 curl -s http://localhost:3001/health/ready | jq '.checks.redis'
```

```
2 # Expected: "ok"
```

## 3.6 Frontend (Next.js) Runbook

### 3.6.1 Common Failure Symptoms

| Symptom                                   | Probable cause                                   |
|-------------------------------------------|--------------------------------------------------|
| Blank page or 500 error on all pages      | Web container crashed; build failed              |
| API calls fail (network error in browser) | NEXT_PUBLIC_API_BASE_URL incorrect; CORS error   |
| Monaco Editor fails to load               | CDN unavailable; CSP header blocking             |
| Pages load but data does not appear       | API returning errors; TanStack Query cache stale |

### 3.6.2 Diagnostic Procedure

```
1 docker compose ps web
```

```
2 docker compose logs web | tail -50
```

```
3
```

```
4 # Test that web container is serving
```

```
5 curl -s -o /dev/null -w "%{http_code}" http://localhost:3000/
```

```
6 # Expected: 200 or 307
```

```
7
```

```
8 # Check that API base URL is baked in correctly
```

```
9 docker compose exec web env | grep NEXT_PUBLIC_API_BASE_URL
```

### 3.6.3 Resolution Steps

Web container crashed:

```
1 docker compose up -d web
```

```
2 docker compose logs -f web
```

Incorrect API base URL (baked at build time):

```
1 # Update .env or docker-compose.yml build arg
```

```
2 # NEXT_PUBLIC_API_BASE_URL is a build-time argument for the web
```

```
3 # image
```

```
3 docker compose up --build -d web
```

## Chapter 4. Common Cross-Cutting Issues

### 4.1 Service Startup Failures

**CRITICAL** — All containers fail to start after deployment.

Diagnosis:

```
1 docker compose up 2>&1 | head -100
2 docker compose logs 2>&1 | grep -E "Error|error|FAILED"
```

Common causes and fixes:

| Cause                            | Fix                                                            |
|----------------------------------|----------------------------------------------------------------|
| .env file missing                | cp .env.example .env<br>and configure                          |
| Port already in use (5432, 6379) | Stop conflicting process:<br>sudo systemctl stop<br>postgresql |
| Insufficient disk space          | Free disk space; see Section<br>3.4                            |
| Docker daemon not running        | sudo systemctl start<br>docker                                 |
| Image build failure              | Check build logs:<br>docker compose build<br>--no-cache        |

### 4.2 Database Connection Failures or Pool Exhaustion

**HIGH** — API returns 500 on all database-touching endpoints. Readiness probe shows database: "error".

Diagnosis:

```
1 # Check postgres is running
2 docker compose ps postgres
3
4 # Check API logs for the exact error
5 docker compose logs api | grep -E "Prisma|database|ECONNREFUSED"
6
7 # Test connectivity from API container
8 docker compose exec api sh -c 'nc -zv postgres 5432 && echo OK'
```

Resolution:

1. Restart PostgreSQL if not running: `docker compose up -d postgres`
2. If pool exhausted, follow Section 3.4 (connection pool fix).
3. If DATABASE\_URL is wrong, fix `.env` and restart: `docker compose up -d api worker`

## 4.3 Authentication / Authorization Failures

**MEDIUM** — Users receive 401 or 403 on previously working requests.

**401 Unauthorized** – all users:

```

1 # Sessions may have all expired or been purged
2 psql $DATABASE_URL -c "SELECT count(*) FROM \"Session\" WHERE \"
   expiresAt\" > NOW();"
3 # If 0: sessions were purged; users must log in again (expected
   behaviour)
4
5 # Check cookie domain/secure settings match the deployment URL
6 docker compose exec api env | grep -E 'AUTH_COOKIE|AUTH_SESSION'
7
8 # If AUTH_COOKIE_SECURE=true but serving over HTTP, sessions will
   not be set

```

**403 Forbidden** – specific endpoint:

```

1 # Check user role
2 psql $DATABASE_URL -c \
3   "SELECT email, role FROM \"User\" WHERE email = 'user@example.
   com';"
4 # Professor-only endpoints require role = 'PROFESSOR'
5 # Student endpoints require role = 'STUDENT'

```

## 4.4 Performance Degradation or Timeouts

**MEDIUM** — API responses are slow; uploads time out; comparison runs take much longer than expected.

**Diagnosis:**

```

1 # Check system resources
2 docker stats --no-stream
3
4 # Check for long-running database queries
5 psql $DATABASE_URL -c \
6   "SELECT pid, now() - query_start AS duration, state, LEFT(query
   ,100)
7   FROM pg_stat_activity WHERE state != 'idle'
8   ORDER BY duration DESC LIMIT 5;"
9
10 # Check BullMQ queue depths

```

```

11 docker compose exec redis redis-cli LLEN bull:comparison-run:
    active
12
13 # Check engine CPU usage during comparison run
14 docker stats worker --no-stream --format "{{.CPUPerc}}"
```

**Resolution:**

- If database is slow: run `VACUUM ANALYZE` (Section 3.4).
- If engine is slow and CPU is maxed: expected behaviour for large assignments. No immediate fix; plan for more CPU or reduce `WORKER_COMPARISON_CONCURRENCY` to 1 to avoid competing for CPU.
- If connection pool exhausted: follow Section 4.2.
- If disk I/O is high: check object storage volume; consider moving to faster storage.

## 4.5 Memory Leaks or Resource Exhaustion

**HIGH** — Worker or API container consumes increasing memory until OOM-killed.

**Diagnosis:**

```

1 # Monitor container memory over time
2 docker stats api worker
3
4 # Check if container was OOM-killed
5 docker inspect $(docker compose ps -q worker) | \
6   jq '.[0].State.OOMKilled'
```

**Resolution:**

- Worker: large submissions accumulate in memory during extraction. Check `ARCHIVE_MAX_EXPANDED_BYTES` is not too high. Restart worker to recover memory.
- Engine: the engine processes one job at a time and exits. Memory is released per invocation. If memory grows, check the `stdout` buffer accumulation in `engine-runner.service.ts` – very large engine outputs are buffered in `stdout` string.
- Set Docker memory limits in `docker-compose.yml`:

```

1 services:
2   worker:
3     deploy:
4       resources:
5         limits:
6           memory: 2G
```

## 4.6 Failed Background Jobs or Scheduled Tasks

**MEDIUM** — Upload batches remain in RECEIVED or PROCESSING; comparison runs remain QUEUED or RUNNING indefinitely.

### Diagnosis:

```

1 # Check failed job counts
2 docker compose exec redis redis-cli LLEN bull:upload-preparation:
   failed
3 docker compose exec redis redis-cli LLEN bull:comparison-run:
   failed
4
5 # Check database for failed batches and runs
6 psql $DATABASE_URL -c \
7     "SELECT id, status, \"errorMessage\" FROM \"UploadBatch\"
8     WHERE status = 'FAILED' ORDER BY \"createdAt\" DESC LIMIT 5;"
9
10 psql $DATABASE_URL -c \
11     "SELECT id, status, \"errorMessage\" FROM \"ComparisonRun\"
12     WHERE status = 'FAILED' ORDER BY \"createdAt\" DESC LIMIT 5;"

```

### Resolution:

If jobs are permanently in PROCESSING/RUNNING but the worker is not processing them (worker restarted mid-job):

```

1 # Reset stuck upload batches
2 psql $DATABASE_URL -c \
3     "UPDATE \"UploadBatch\" SET status = 'FAILED',
4     \"errorMessage\" = 'Processing interrupted. Please re-upload.'
5     WHERE status = 'PROCESSING';"
6
7 # Reset stuck comparison runs
8 psql $DATABASE_URL -c \
9     "UPDATE \"ComparisonRun\" SET status = 'FAILED',
10     \"errorMessage\" = 'Run was interrupted. Please try again.'
11     WHERE status = 'RUNNING';"

```

Advise the professor to re-upload or re-run the comparison.

## 4.7 Data Corruption or Inconsistency Scenarios

**CRITICAL** — Submission source data is missing or incorrect; pair results reference deleted submissions.

**Prevention:** Prisma wraps multi-table writes in transactions (e.g., in `deleteAssignment`, account deletion). Partial writes are extremely unlikely. Object storage deletions are done outside the transaction with best-effort cleanup.

### Detection:

```

1 # Find UploadBatch records with no Submission children (
   extraction failed silently)

```



```

2  psql $DATABASE_URL -c \
3    "SELECT ub.id, ub.status, ub.\"assignmentId\"
4    FROM \"UploadBatch\" ub
5    WHERE ub.status = 'READY'
6    AND NOT EXISTS (SELECT 1 FROM \"Submission\" s WHERE s.\"
7    uploadBatchId\" = ub.id);"
8
9  # Find Submissions with empty concatenated source
10 psql $DATABASE_URL -c \
11    "SELECT id, \"displayName\", length(\"concatenatedSource\") AS
12    src_len
13    FROM \"Submission\" WHERE length(\"concatenatedSource\") < 10;
14    "
15
16 # Find PairResult rows referencing non-existent submissions
17 psql $DATABASE_URL -c \
18    "SELECT pr.id FROM \"PairResult\" pr
19    WHERE NOT EXISTS (SELECT 1 FROM \"Submission\" s WHERE s.id =
20    pr.\"leftSubmissionId\")
21    OR NOT EXISTS (SELECT 1 FROM \"Submission\" s WHERE s.id =
22    pr.\"rightSubmissionId\");"

```

**Resolution:**

- For missing submissions: professor must re-upload the affected archive.
- For orphaned pair results: delete the affected comparison run and re-run.
- For serious corruption: restore from backup (see Maintenance Manual Section 3.1).

## 4.8 Configuration Errors

**MEDIUM** — Application starts but behaves incorrectly due to wrong environment variable values.

**Diagnosis:**

```

1  # Print all environment variables for each service
2  docker compose exec api env | sort
3  docker compose exec worker env | sort
4
5  # Common misconfigurations:
6  # - CORS_ALLOWED_ORIGINS missing port (e.g., http://localhost
7     instead of http://localhost:3000)
8  # - AUTH_COOKIE_SECURE=true when not serving HTTPS
9  # - NEXT_PUBLIC_API_BASE_URL pointing to wrong host
10 # - ENGINE_BINARY_PATH pointing to wrong location

```

**Resolution:** Edit .env, then:

```

1  docker compose up -d # Picks up new env values

```

## 4.9 Dependency / Integration Failures (Third-Party Services Down)

The application has no direct third-party API dependencies at runtime. All external dependencies are infrastructure (PostgreSQL, Redis). Caddy fetches TLS certificates from Let's Encrypt; if Let's Encrypt is unreachable, certificate renewal fails but existing certificates continue to work until expiry.

Let's Encrypt certificate renewal failure:

```
1 # Check Caddy logs for certificate errors
2 docker compose logs caddy | grep -i "certificate\\|acme\\|tls"
3
4 # Caddy will retry renewal automatically
5 # Force renewal:
6 docker compose exec caddy caddy reload --config /etc/caddy/
   Caddyfile
```

## 4.10 Deployment Failures and Rollback

**HIGH** — A new deployment breaks functionality. Need to roll back.

Rollback procedure:

```
1 # If using Git tags for releases:
2 git log --oneline -10 # Find last good commit
3 git checkout <last-good-tag>
4
5 # Rebuild and redeploy (database migrations are forward-only;
6 # ensure the old code is compatible with the new schema before
   deploying)
7 docker compose up --build -d
8
9 # If the new deployment also ran a database migration that cannot
   be reversed:
10 # Restore from backup (Section 3.1 of Maintenance Manual)
11 # Then deploy the old code version
```

**NOTE:** Always take a database backup before deploying a new version that includes database migrations. Prisma migrations are not automatically reversible.

## Chapter 5. Data Recovery Procedures

### 5.1 Full Restore from Backup

Listing 5.1: Full system restore procedure

```
1  # 1. Stop all services except postgres
2  docker compose stop api worker web caddy
3
4  # 2. Drop and recreate database
5  docker compose exec postgres psql -U postgres <<EOF
6  DROP DATABASE IF EXISTS similarity_app;
7  CREATE DATABASE similarity_app;
8  EOF
9
10 # 3. Restore database dump
11 docker compose exec -T postgres pg_restore \
12     -U postgres \
13     -d similarity_app \
14     < ./backups/similarity_app_YYYYMMDD_HHMMSS.dump
15
16 # 4. Restore object storage
17 docker run --rm \
18     -v similarity-app-object-storage:/target \
19     -v $(pwd)/backups:/backup \
20     busybox sh -c "rm -rf /target/* && tar xzf /backup/object-
21         storage_YYYYMMDD.tar.gz \
22         -C /target --strip-components=1"
23
24 # 5. Apply any migrations that were added after the backup
25 npm run db:migrate:deploy
26
27 # 6. Restart all services
28 docker compose up -d
29
30 # 7. Verify
31 curl -s http://localhost/api/health/ready | jq .
```

### 5.2 Point-in-Time Recovery

**NOTE:** Point-in-time recovery (PITR) is not configured. The default PostgreSQL setup does not enable WAL archiving. PITR requires enabling `wal_level = replica` and archiving WAL segments to an external location. Without this, recovery is limited to the most recent dump.

To enable WAL archiving (add to PostgreSQL configuration):

```
1 wal_level = replica
2 archive_mode = on
3 archive_command = 'cp %p /wal-archive/%f'
```

## 5.3 Partial Data Recovery

To recover a single deleted submission (if a backup exists):

Listing 5.2: Partial submission recovery

```
1 # 1. Restore backup to a temporary database
2 docker compose exec postgres psql -U postgres -c \
3 "CREATE DATABASE similarity_app_restore;"
4 docker compose exec -T postgres pg_restore \
5 -U postgres -d similarity_app_restore \
6 < ./backups/similarity_app_YYYYMMDD.dump
7
8 # 2. Extract the specific submission
9 psql "postgres://postgres:postgres@localhost:5432/
10 similarity_app_restore" -c \
11 "SELECT * FROM \"Submission\" WHERE \"displayName\" = '<student
12 -name>';" \
13 > recovered_submission.txt
14
15 # 3. Re-insert into live database if still needed
16 # (manual SQL INSERT based on recovered data)
17
18 # 4. Drop temporary database
19 docker compose exec postgres psql -U postgres -c \
20 "DROP DATABASE similarity_app_restore;"
```

## Chapter 6. Post-Incident Procedures

### 6.1 Root Cause Analysis Template

After resolving any High or Critical incident, complete a root cause analysis (RCA) within 48 hours.

Listing 6.1: Root cause analysis template

```
1 Incident ID:      INC-YYYY-NNN
2 Date/Time:       YYYY-MM-DD HH:MM UTC
3 Severity:        Critical / High / Medium
4 Duration:        X hours Y minutes
5 Reported by:     Name / role
6 Resolved by:     Name / role
7
8 Summary:
9   One-sentence description of the incident.
10
11 Timeline:
12   HH:MM - Incident first detected / reported
13   HH:MM - Investigation started
14   HH:MM - Root cause identified
15   HH:MM - Fix applied
16   HH:MM - Service restored
17   HH:MM - Post-incident monitoring confirmed stable
18
19 Impact:
20   - Which components were affected?
21   - How many users / assignments / comparison runs were impacted?
22   - Was any data lost or corrupted?
23
24 Root Cause:
25   Describe the actual technical cause (not just symptoms).
26
27 Contributing Factors:
28   - What conditions allowed this to occur?
29   - Was there missing monitoring, testing, or documentation?
30
31 Resolution:
32   What was done to restore service?
33
34 Corrective Actions:
35   Item 1: [action] -- Owner: [name] -- Due: [date]
36   Item 2: [action] -- Owner: [name] -- Due: [date]
37
38 Lessons Learned:
39   What should be done differently in the future?
```

## 6.2 Incident Report

For incidents affecting end users, send a brief incident report to affected stakeholders within 24 hours of resolution:

Listing 6.2: Incident report template

```
1 Subject: [RESOLVED] Similarity App Incident - YYYY-MM-DD
2
3 We experienced a service disruption on [DATE] from [START TIME]
  to [END TIME] (UTC).
4
5 Affected functionality: [description]
6
7 Root cause: [one paragraph non-technical explanation]
8
9 What we did to fix it: [brief description]
10
11 What we are doing to prevent recurrence: [brief description]
12
13 We apologise for any disruption to your work.
```

## 6.3 Implementing Preventive Fixes

After each incident, the corrective actions identified in the RCA should be tracked to completion. Recommended categories:

- **Monitoring:** Add alerts for the metrics that would have caught this incident earlier (see Maintenance Manual Section 3.6).
- **Runbook update:** Update this document with the new failure mode and its resolution.
- **Code fix:** If a bug in the application caused the incident, create a ticket and fix it.
- **Infrastructure hardening:** If a configuration gap enabled the incident (e.g., missing rate limiting, no connection pool limit), implement the hardening.
- **Backup validation:** If data recovery was needed, validate that backup/restore procedures work correctly by doing a restore drill.